

```

64 var x = o.left;
65 var y = o.top;
66
67 var ax = settings.accX;
68 var ay = settings.accY;
69 var th = t.height();
70 var wh = w.height();
71 var tw = t.width();
72 var ww = w.width();
73
74 if (y + th + ay >= b &&
75     y <= b + wh + ay &&
76     x + tw + ax >= a &&
77     x <= a + ww + ax) {
78     //trigger the custom event
79     if (!t.appeared) t.trigger('appear', settings.data);
80
81     } else {
82     //it scrolled out of view
83     t.appeared = false;
84     }
85 };
86
87 //create a modified fn with some additional logic
88 var modifiedFn = function() {
89
90     //mark the element as visible
91     t.appeared = true;
92
93     //is this supposed to happen only once?
94     if (settings.one) {
95
96         //remove the check
97         w.unbind('scroll', check);
98         var i = $.inArray(check, $.fn.appear.checks);
99         if (i >= 0) $.fn.appear.checks.splice(i, 1);
100     }
101
102     //trigger the original fn
103     fn.apply(this, arguments);
104
105     //bind the modified fn to the element
106     //if (settings.one) t.one('appear', settings.data, modifiedFn);
107     t.on('appear', settings.data, modifiedFn);
108 };

```



MATERIA:

Programación Orientada a Objetos

Unidad 3 Programación Orientada a Objetos en Desarrollo Moderno

Docente:

Grupo:

Unidad #3 - Programación Orientada a Objetos en Desarrollo Moderno.

3.11 Principios de arquitectura limpia en Spring Boot.

3.12 Separación en capas: controlador, servicio, repositorio.

3.13 Streams y Lambdas

3.14 Uso de Spring Data JPA.

3.15 Conexión con base de datos H2 (en memoria).

3.16 Anotaciones: @Entity, @Id, @GeneratedValue,
@Repository.

3.11 Principios de Arquitectura Limpia

El Fundamento: Separar Responsabilidades

El objetivo es que la lógica de negocio (el núcleo) no dependa de detalles externos como frameworks o bases de datos.

Regla de la Dependencia: Las dependencias solo apuntan hacia adentro.

Fácil de Probar: La lógica de negocio se puede probar de forma aislada.

Intercambiable: Puedes cambiar de MySQL a MongoDB sin tocar la lógica de negocio.



3.12 Separación en Capas

Es la implementación más popular de la Arquitectura Limpia.

Controlador (@RestController)

Punto de entrada de la API. Recibe peticiones HTTP.

NO contiene lógica de negocio. Delega el trabajo al Servicio.

Servicio (@Service)

El cerebro de la aplicación. Contiene la lógica de negocio (validaciones, cálculos).
Orquesta las llamadas a los repositorios.

Repositorio (@Repository)

Abstracción de la base de datos. Se encarga únicamente del acceso a los datos (CRUD).

Ejemplo: Crear un Producto

POST /api/productos

Cliente envía un JSON (DTO) al **Controlador**.

El **Controlador** recibe el DTO y llama a `productoService.crearProducto(dto)`.

El **Servicio** valida la lógica de negocio (ej. el SKU no existe), convierte el DTO a una Entidad Producto.

El **Servicio** llama a `productoRepository.save(producto)`.

El **Repositorio** (implementado por Spring Data JPA) guarda el producto en la base de datos.

La respuesta viaja de vuelta al cliente.

3.13 Streams y Lambdas

Procesando Datos de Forma Moderna

Introducidos en Java 8 para escribir código más declarativo y legible.

Un **Stream** en Java es una secuencia de datos que puedes procesar con operaciones funcionales (sin bucles explícitos).

Y las **Lambdas** son funciones anónimas que permiten escribir ese procesamiento de forma compacta.

Tipos de lambdas

Sin parámetros

```
() -> System.out.println("Hola mundo")
```

Runnable

Con un parámetro

```
x -> x * x
```

Function<T, R>

Con varios parámetros

```
(a, b) -> a + b
```

BinaryOperator<T>

Con cuerpo de varias líneas

```
(x, y) -> { int suma = x + y;  
            return suma * 2;
```

BinaryOperator<T>

Streams y Lambdas

Lambda (->): Una función anónima y corta.

producto -> producto.getPrecio() > 100

Stream: Una "tubería" para procesar colecciones de datos.

Ejemplo: Obtener nombres de productos en oferta

Los métodos `map()`, `filter()` y `sorted()` son **operaciones intermedias** del API de **Streams en Java**.

Sirven para **procesar colecciones** de manera declarativa y funcional, sin usar bucles explícitos (`for`, `while`, etc.).

```
List<String> ofertas = productos.stream()           // 1. Crear Stream
    .filter(p -> p.getPrecio() < 50.0)           // 2. Filtrar por precio
    .map(Producto::getNombre)                     // 3. Mapear a nombre
    .collect(Collectors.toList());                // 4. Recolectar en una lista
```

🧩 Resumen visual

Método	Qué hace	Tipo de Lambda	Ejemplo
<code>filter()</code>	Filtra elementos según condición	<code>Predicate<T></code>	<code>n -> n > 5</code>
<code>map()</code>	Transforma elementos	<code>Function<T, R></code>	<code>n -> n * 2</code>
<code>sorted()</code>	Ordena elementos	<code>Comparator<T></code>	<code>(a, b) -> a - b</code>

3.14 Uso de Spring Data JPA

Spring Data JPA simplifica al máximo la capa de persistencia.

JpaRepository<Producto, Long>: Solo necesitas definir una interfaz. Spring crea la implementación con métodos CRUD listos para usar (save, findById, findAll, delete).

Consultas Derivadas: Spring escribe el SQL por ti a partir del nombre del método.

```
@Repository
public interface ProductoRepository extends JpaRepository<Producto, Long> {

    // Spring entiende esto y crea:
    // SELECT * FROM productos WHERE sku = ?
    Optional<Producto> findBySku(String sku);

    // SELECT * FROM productos WHERE categoria = ? AND stock > ?
    List<Producto> findByCategoriaAndStockGreaterThan(String cat, int minStock);
}
```

3.15 Conexión a Bases de Datos

Spring Boot facilita el cambio de base de datos modificando solo la configuración.

1. H2 (En Memoria)

Ideal para desarrollo y pruebas. No requiere instalación.

- **pom.xml:** `h2`
- **application.properties:** `spring.datasource.url=jdbc:h2:mem:tiendadb`

2. MySQL (JDBC)

Para entornos de producción robustos.

- **pom.xml:** `mysql-connector-j`
- **application.properties:** `spring.datasource.url=jdbc:mysql://localhost:3306/tienda_db`

3. MongoDB (NoSQL)

Para datos flexibles. Usa `MongoRepository`.

- **pom.xml:** `spring-boot-starter-data-mongodb`
- **application.properties:** `spring.data.mongodb.uri=mongodb://localhost:27017/tienda_mongo_db`

3.16 Anotaciones Clave

Mapeando Clases a Tablas

Estas anotaciones le dicen a JPA cómo persistir tus objetos.

- `@Entity`: Marca una clase como una tabla de la base de datos.
- `@Id`: Define la clave primaria.
- `@GeneratedValue`: Configura la auto-generación del ID (ej. `AUTO_INCREMENT`).
- `@Repository`: Define un bean de Spring para el acceso a datos.

```
// Ejemplo de Entidad Producto
@Data
@Entity
@Table(name = "productos")
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String sku;

    private String nombre;
    private double precio;
}
```